

ELE 402 - GRADUATION PROJECT II FINAL REPORT

HACETTEPE UNIVERSITY
DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

PROJECT TITLE: Rubik's Cube Solver

PROJECT GROUP MEMBERS: Özge ERDEM – Görkem YÜKSEL

PROJECT SUPERVISOR: ASSOC. PROF. DR. BARIŞ YÜKSEKKAYA

SUBMISSION DATE: 14.04.2026

SPRING 2025-2026

ABSTRACT

This final report presents the design, implementation, and testing of the Rubik's Cube Solver, an autonomous robotic system developed within ELE 401 and ELE 402. The system scans a scrambled 3 x 3 x 3 Rubik's Cube using a camera, classifies sticker colors with an HSV-based OpenCV pipeline, computes a solution with the Kociemba two-phase algorithm, and physically executes the moves using six NEMA 17 stepper motors. During ELE 401, the software foundation and solver logic were created; during ELE 402, the project was completed as an integrated Raspberry Pi 4B-based prototype with a custom graphical user interface, a six-motor mechanical frame, and a custom PCB for A4988 motor driver synchronization and power distribution. After the interim report, the final version added the graphical interface, full scan-solve-execute synchronization, and a Random Mix feature that scrambles the cube, clears the stored cube state and previous solution, and forces a new solve operation when the Solve command is pressed. The final design demonstrates an end-to-end autonomous solution pipeline and provides a low-cost educational platform for robotics, computer vision, embedded systems, and motion planning.

CONTENTS

ABSTRACT	1
LIST OF FIGURES AND TABLES	3
1. INTRODUCTION	4
3. ENGINEERING STANDARDS AND DESIGN CONSTRAINTS	6
3.1. ENGINEERING STANDARDS.....	6
3.2. DESIGN CONSTRAINTS	6
4. SUSTAINABLE DEVELOPMENT GOALS	7
5. LITERATURE REVIEW	8
6. METHODS	9
6.1. CAMERA-BASED COLOR ACQUISITION AND GUI VERIFICATION	9
6.2. KOCIEMBA SOLVER AND CUBE STATE ENCODING	10
6.3. SIX-MOTOR ACTUATION AND PCB-BASED DRIVER CONTROL.....	10
6.4. RANDOM MIX AND STATE RESET METHOD	11
7. PRELIMINARY DESIGN	12
8. PROTOTYPE.....	13
9. DESIGN PROCESS AND PROGRESS.....	14
9.1. ITERATION 1: SOFTWARE PIPELINE AND COLOR DETECTION	14
9.1.1. TESTING AND RESULTS.....	15
9.1.2. EVALUATION	15
9.2. ITERATION 2: MECHANICAL FRAME, PCB, AND MOTOR CONTROL	16
9.2.1. TESTING AND RESULTS.....	16
9.2.2. EVALUATION	17
9.3. ITERATION 3: FINAL GUI, RANDOM MIX, AND END-TO-END OPERATION	17
9.3.1. TESTING AND RESULTS.....	17
9.3.2. EVALUATION	17
10. FINAL DESIGN	18
10.1. MEETING THE CONSTRAINTS AND ENGINEERING STANDARDS.....	19
10.2. COST ANALYSIS	20
11. TEAMWORK.....	21
12. COMMENTS AND CONCLUSIONS	22
REFERENCES	23

LIST OF FIGURES AND TABLES

Figure 1. Final electromechanical architecture of the Rubik's Cube Solver	5
Figure 2. Camera-based color acquisition pipeline	9
Figure 3. Mechanical system and final six-motor prototype	10
Figure 4. Preliminary system architecture from the interim design	12
Figure 5. Custom graphical user interface used for live state display and manual correction	13
Figure 6. OpenCV color detection and ROI mapping on the Raspberry Pi	14
Figure 7. Custom PCB layout and transition from breadboard wiring to centralized motor control	16
Figure 8. Final state flowchart with Random Mix and state reset behavior	18
Figure 9. Standard Rubik's Cube move notation used by the solver and motor parser	19
Table 1. Main final design requirements and constraints	5
Table 2. Random Mix feature behavior and validation points	11
Table 3. Estimated cost analysis of the final prototype	20

1. INTRODUCTION

The Rubik's Cube Solver project aims to build an autonomous robotic system that can solve a standard 3 x 3 x 3 Rubik's Cube without manual intervention after the cube is placed into the mechanism. The final system combines computer vision, embedded control, motor driver electronics, mechanical design, and a graphical user interface. The overall goal is to convert a visually observed scrambled cube into a valid cube-state string, generate a move sequence using the Kociemba two-phase algorithm, and physically rotate the cube faces with six independent stepper motors.

In ELE 401, the project mainly focused on the software concept: the cube-state representation, initial color detection trials, solver integration, and a mobile-application-oriented workflow. In ELE 402, the project moved from a conceptual and software-only state into an integrated physical prototype. The architecture was redesigned around a Raspberry Pi 4B so that image acquisition, OpenCV processing, solving, GUI control, and motor commands could run locally on a single embedded platform.

The interim report documented two major design iterations: the first iteration established the Raspberry Pi software pipeline with OpenCV and Kociemba, while the second iteration focused on the custom PCB, stepper motor actuation, and 3D-printed mechanical frame. In the final period, the system was further improved with a functional GUI, synchronized scan-solve-execute operation, and a Random Mix feature. In this feature, the robot randomly scrambles the cube, intentionally clears the saved cube state and previous solution, and waits for a new Solve command. Therefore, the system does not simply reverse the random sequence; it scans and solves the newly mixed cube again. This final report presents the complete project description, standards, constraints, methods, design iterations, final design, cost analysis, teamwork, and conclusions.

2. PROJECT DESCRIPTION

The project is a mechatronic Rubik's Cube solving robot. The user places a cube inside the mechanical frame and starts the operation from the GUI. The camera captures cube-face images, and the software detects the sticker colors using HSV-based segmentation. The detected colors are displayed in the GUI so that the user can manually correct possible detection errors before solving. After a valid cube state is obtained, the Kociemba solver generates a sequence of moves such as U, R', F2, B, L', or D2. The motor-control layer parses these moves and sends step and direction signals to A4988 drivers, which control six NEMA 17 stepper motors, one motor for each cube face.

The final prototype uses a Raspberry Pi 4B as the central controller. It coordinates the camera, OpenCV scripts, GUI, solver, GPIO outputs, motor drivers, and state memory. The custom PCB reduces wiring complexity by centralizing A4988 driver connections, Raspberry Pi GPIO connections, power distribution, and common ground routing. The 3D-printed frame holds the cube in a fixed position and aligns the six motors with the six cube faces. This architecture supports automatic solving, GUI-based manual verification, and the newly added Random Mix command.

Table 1. Main final design requirements and constraints

Requirement / Constraint	Final implementation
Autonomous cube solving	The system performs scan, GUI verification, Kociemba solving, move parsing, and motor execution on the Raspberry Pi.
Six-face actuation	Six NEMA 17 motors are driven through A4988 drivers; one motor is assigned to each cube face.
Solution efficiency	The solver uses the Kociemba two-phase algorithm, targeting solutions under 20 moves for the final demonstration.
Real-time user visibility	The GUI displays the live cube state and allows manual correction before solving.
Random Mix validation	Random Mix scrambles the cube with motor commands, clears stored state, and requires a new solve operation.
Low-cost prototype	The design relies on accessible components: Raspberry Pi, A4988 drivers, NEMA 17 motors, 3D-printed parts, and a custom PCB.

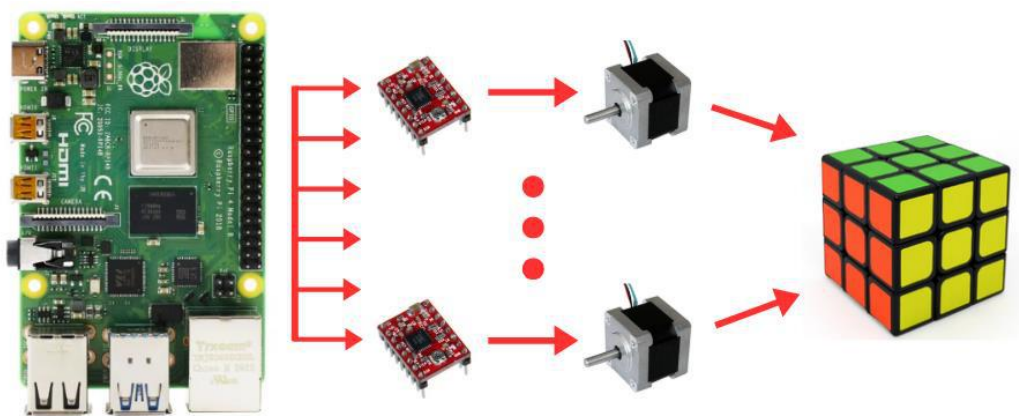


Figure 1. Final electromechanical architecture of the Rubik's Cube Solver

3. ENGINEERING STANDARDS AND DESIGN CONSTRAINTS

3.1. ENGINEERING STANDARDS

The final design is a university prototype and is not claimed to be formally certified. However, the design process follows relevant engineering standards and good-practice principles in software, electronics, and mechatronic implementation.

IPC-2221, Generic Standard on Printed Board Design, is relevant to the custom PCB because the board includes motor driver modules, power-distribution paths, connector placement, and signal routing between the Raspberry Pi and six A4988 drivers. In accordance with this design logic, high-current motor paths were separated from low-current GPIO signals as much as possible, connectors were organized to reduce wiring errors, and the PCB was used to improve reliability compared with the breadboard prototype.

IEC 62368-1 safety principles are relevant because the prototype combines a low-voltage computing unit, external motor power, and moving mechanical parts. Although formal product certification was outside the scope of the graduation project, the final design separates logic-level control from motor power, uses a common ground reference, avoids exposed high-voltage circuitry, and keeps the moving cube mechanism inside a fixed frame during operation.

ISO/IEC/IEEE 12207 software life-cycle principles are also applicable at the project level. The final software was divided into color acquisition, GUI verification, solver, random-mix generation, state-memory management, move parsing, and motor-control modules. This modular structure made debugging and final integration easier.

3.2. DESIGN CONSTRAINTS

The project was constrained by budget, component availability, time, mechanical precision, lighting conditions, and embedded computation. The robot had to be realizable with parts that could be obtained and assembled within the ELE 402 schedule. The six-motor frame had to fit a standard 3 x 3 x 3 cube while leaving enough clearance for face rotations. The stepper motors had to rotate cube faces by 90 degrees and 180 degrees with sufficiently low angular error. On the software side, the color detection method had to be fast enough to run on the Raspberry Pi and robust enough for the GUI to create a valid cube-state string. The final GUI correction step was therefore included as a practical constraint-management method: a wrong color can be corrected before the solver is run.

4. SUSTAINABLE DEVELOPMENT GOALS

The project supports several Sustainable Development Goals at the prototype and educational level. Goal 4, Quality Education, is the strongest match because the robot is a hands-on learning platform for computer vision, embedded systems, motor control, printed-circuit-board design, and mechanical integration. Students can observe how a real mechatronic system moves from algorithmic planning to physical actuation.

Goal 9, Industry, Innovation and Infrastructure, is also supported because the project integrates a custom PCB, a multi-motor actuation structure, and an embedded perception-and-control pipeline. The architecture is directly related to broader automation tasks such as pick-and-place systems, sorting mechanisms, and low-cost robotic testbeds. Goal 12, Responsible Consumption and Production, is addressed by using 3D-printed replaceable parts and a modular PCB-based architecture, which makes maintenance and component replacement easier than rebuilding the whole system.

5. LITERATURE REVIEW

A Rubik's Cube state can be described by the colors or cubie positions of its 54 stickers. Standard cube notation defines face moves U, D, F, B, R, and L, along with prime moves for counter-clockwise rotation and the suffix 2 for 180-degree rotation. This notation is suitable both for solver output and for motor-command parsing because each symbolic move can be mapped directly to one motor, one direction, and one angle.

The selected solving method is the Kociemba two-phase algorithm. Compared with a brute-force search, Kociemba's approach reduces the solution search space by dividing the problem into phases and using precomputed move tables. This makes it practical to run the solver on an embedded platform such as a Raspberry Pi. The final project therefore uses Kociemba output as the bridge between visual cube-state recognition and physical motor control [1], [2].

For perception, OpenCV is used because it provides efficient image acquisition, color-space conversion, filtering, and region-of-interest operations. HSV color representation was preferred over direct RGB comparison because hue, saturation, and value provide a more intuitive separation of sticker colors under moderate lighting variation. Nevertheless, the interim tests showed that red/orange overlap and reflections can still cause errors, so the final system includes a GUI correction stage before solving [4].

For actuation, stepper motors were selected because they provide discrete angular positioning without requiring a complex feedback controller for a first prototype. A4988 motor driver modules receive step and direction signals from the Raspberry Pi GPIO pins and control the phase currents of the NEMA 17 motors. This structure is simple, widely documented, and suitable for 90-degree and 180-degree cube-face rotations [5], [6].

The literature and project examples on Rubik's Cube solving robots show that successful systems require not only a solver algorithm but also reliable synchronization between scanning, state validation, and mechanical execution. For this reason, the final design emphasizes the full scan-solve-execute loop, GUI verification, and state-reset behavior after random scrambling [7].

6. METHODS

6.1. CAMERA-BASED COLOR ACQUISITION AND GUI VERIFICATION

The first method is camera-based color acquisition. The camera captures images of cube faces, and each face is divided into nine fixed regions of interest. The mean HSV values of these regions are calculated and classified into the six cube colors: white, yellow, red, orange, blue, and green. The GUI then visualizes the detected state. If one or more stickers are misclassified, the user can manually correct them before the solver is called. This approach was selected because it combines automation with practical robustness: the system remains usable even when lighting conditions are not ideal.

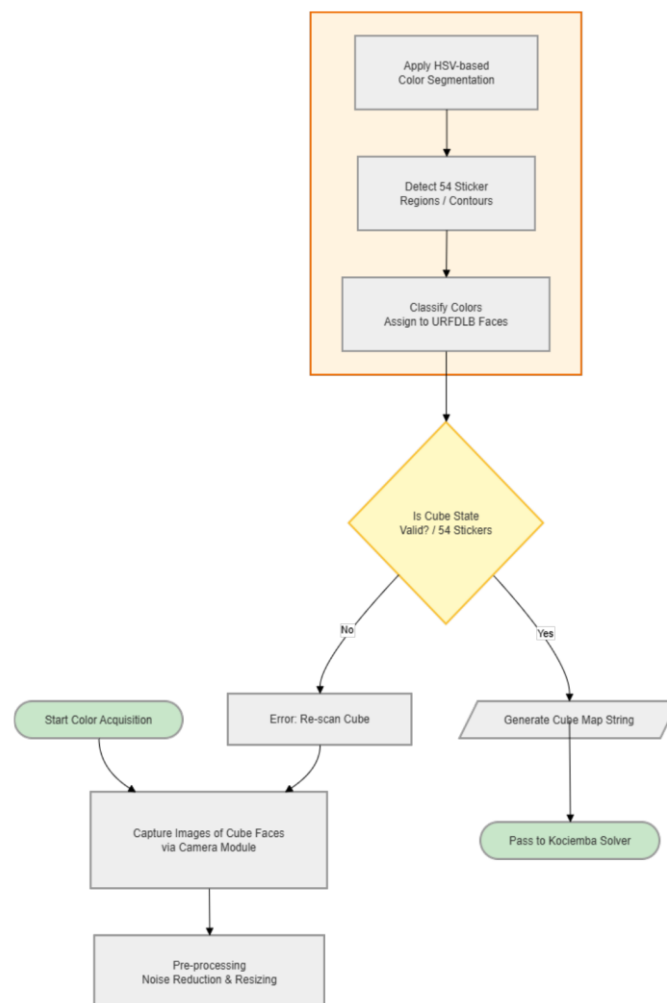


Figure 2. Camera-based color acquisition pipeline

6.2. KOCIEMBA SOLVER AND CUBE STATE ENCODING

The second method is solver-based motion planning. After the GUI confirms that the cube state is valid, the 54 stickers are converted into the face-order required by the solver. The Kociemba module returns a move list in standard notation. The motor parser converts each move into motor index, direction, and number of steps. A move without a suffix corresponds to a 90-degree clockwise rotation; a prime move corresponds to a 90-degree counter-clockwise rotation; a move with the suffix 2 corresponds to a 180-degree rotation.

6.3. SIX-MOTOR ACTUATION AND PCB-BASED DRIVER CONTROL

The third method is six-motor face-centered actuation. One NEMA 17 stepper motor is assigned to each cube face. A custom PCB holds or connects the A4988 motor driver modules and distributes power and GPIO signals. The PCB was selected instead of a fully breadboarded system because the final design requires many repeated motor-driver connections. Centralized wiring reduces connection errors and improves demonstration reliability.

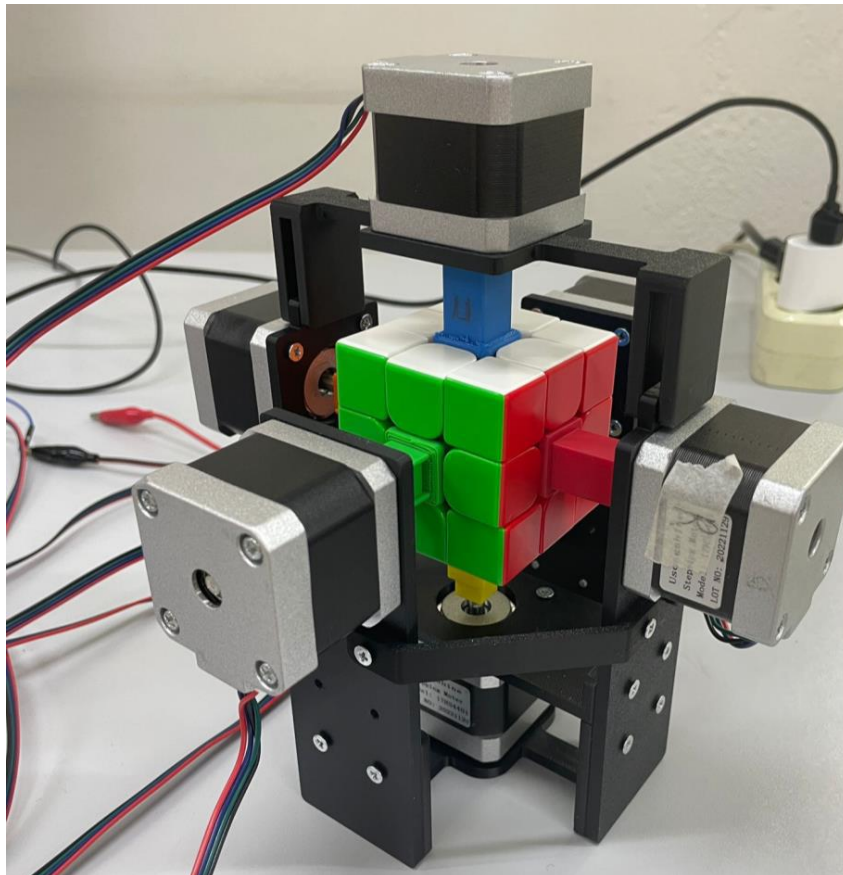


Figure 3. Mechanical system and final six-motor prototype

6.4. RANDOM MIX AND STATE RESET METHOD

The final method added after the interim report is the Random Mix and state reset method. When the user presses Random Mix, the software generates a random legal sequence of cube moves and physically executes this sequence with the motors. After the sequence is executed, the software clears the stored cube-state string and the previous solution list. Therefore, when the user presses Solve, the program cannot simply apply the inverse of the random scramble. It must capture or verify the current cube state again, run the Kociemba solver again, and generate a new solution. This feature validates the autonomy of the complete system.

Table 2. Random Mix feature behavior and validation points

Random Mix step	Purpose in final design
Generate random move sequence	Creates a new physical scramble without requiring the user to hand-mix the cube.
Execute sequence with motors	Verifies that the motor-control layer can perform arbitrary face rotations.
Clear previous state and solution	Prevents the software from solving by reversing a stored scramble sequence.
Wait for Solve command	Forces a new scan/verification/solve operation on the mixed cube.
Run solver again	Demonstrates end-to-end autonomy of the final design.

7. PRELIMINARY DESIGN

The preliminary design from ELE 401 was a software-centered structure that combined mobile-app interaction with Python-based solving. The core idea was to obtain the cube colors, send the cube state to a solver, and present the solution to the user. However, this design had two major limitations for the final project: it relied on a separate application pipeline, and it did not yet include a complete physical actuation system.

The preliminary ELE 402 redesign replaced the mobile-oriented workflow with a Raspberry Pi-based standalone workflow. The Raspberry Pi was selected as the central controller because it can run Python, OpenCV, the Kociemba solver, a GUI, and GPIO-based motor-control scripts. The preliminary hardware design consisted of a camera module, a six-motor mechanical frame, A4988 drivers, a custom PCB, and a separate motor power supply. The main design parameters were camera position, ROI size, HSV thresholds, cube-face order, motor step count per 90-degree turn, motor speed delay, driver current limit, power distribution, GUI state representation, and emergency stop behavior.

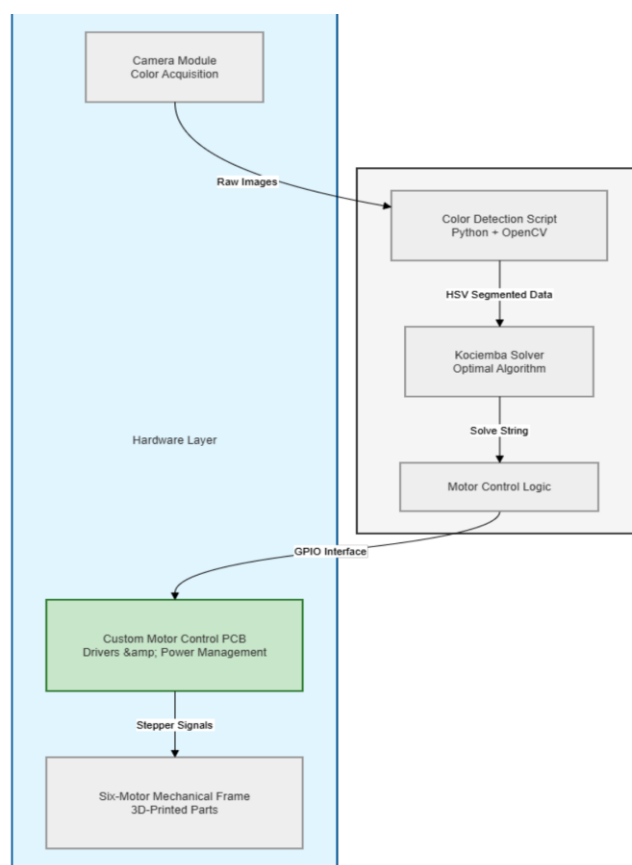


Figure 4. Preliminary system architecture from the interim design

8. PROTOTYPE

The final prototype is a hardware/software integrated Rubik's Cube solving robot. The prototype was built to validate the complete scan-solve-execute sequence in a standalone configuration. Its key features are listed below.

- Raspberry Pi 4B central controller running Python, OpenCV, the GUI, Kociemba solver, random-mix logic, and GPIO motor commands.
- Camera-based color acquisition with HSV segmentation and nine ROIs per cube face.
- Graphical user interface displaying the live cube state and allowing manual color correction before solving.
- Six NEMA 17 stepper motors placed around the cube, one motor per face.
- Six A4988 motor drivers coordinated through Raspberry Pi GPIO pins and a custom PCB.
- 3D-printed frame and cube-face adapters that hold the cube and transfer motor motion to the faces.
- Random Mix feature that scrambles the cube, clears stored state, and requires a fresh solve operation.

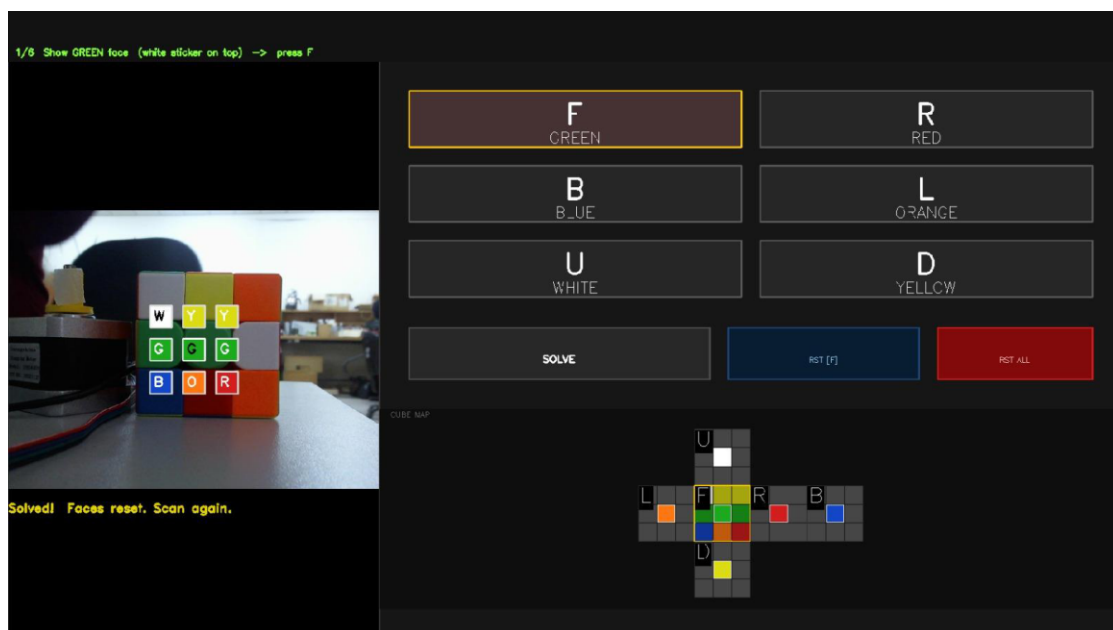


Figure 5. Custom graphical user interface used for live state display and manual correction

9. DESIGN PROCESS AND PROGRESS

The design process was iterative. Each iteration improved a different weakness of the previous version. The first iteration validated the software pipeline, the second iteration added physical motor-control and mechanical implementation, and the third iteration completed the final user workflow with GUI integration, Random Mix, and state-reset logic.

9.1. ITERATION 1: SOFTWARE PIPELINE AND COLOR DETECTION

The first iteration focused on developing a self-contained software pipeline on the Raspberry Pi. The camera captured cube-face images, OpenCV converted the images into HSV color space, fixed ROIs isolated the stickers, and the mean HSV values were mapped to six cube colors. The 54 detected stickers were assembled into the solver input string. At this stage, the execution phase remained theoretical and was displayed in the terminal.

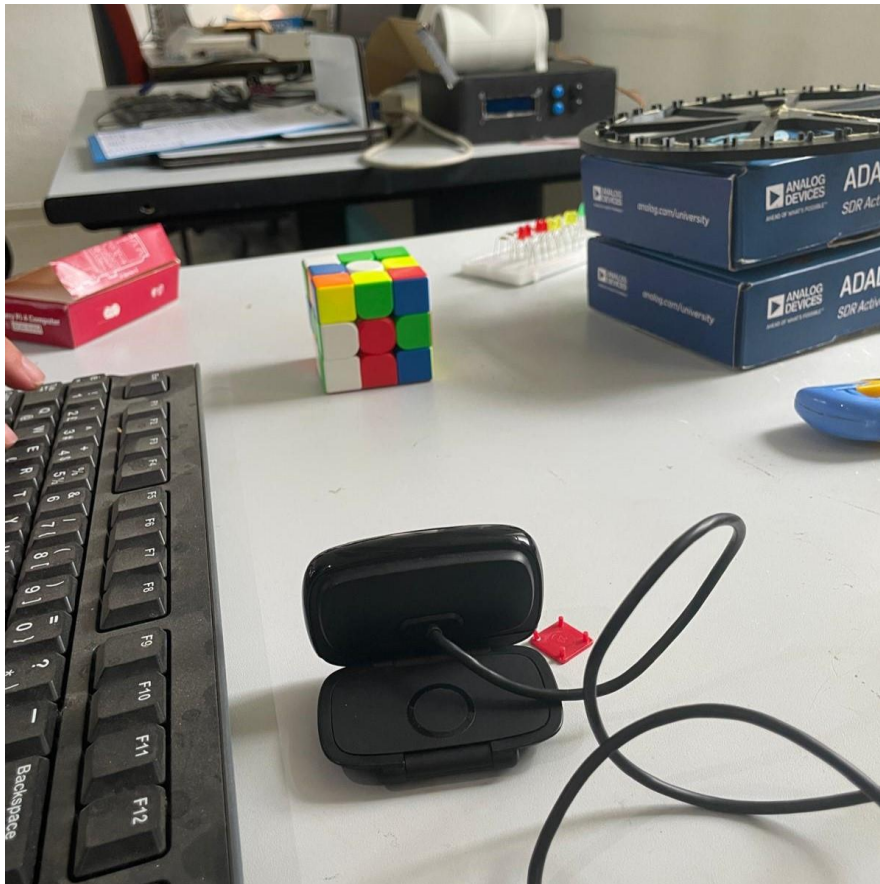


Figure 6. OpenCV color detection and ROI mapping on the Raspberry Pi

9.1.1. TESTING AND RESULTS

The color-detection module was tested under ambient indoor lighting and direct desk-lamp illumination. In the interim tests, ambient indoor lighting produced approximately 52 correct stickers out of 54 on average, corresponding to about 96 percent accuracy. Under direct desk-lamp illumination, the accuracy decreased to about 89 percent because reflections shifted the apparent hue values. The solver was separately tested with valid cube-state strings and produced valid solutions with an average computation time under 0.8 seconds on the Raspberry Pi.

9.1.2. EVALUATION

This iteration validated the software foundation but revealed two main problems. First, fixed HSV thresholds were sensitive to lighting changes, especially in the red/orange boundary region. Second, the system had no physical actuation, so it could not complete the project goal. The next iteration therefore focused on improved threshold tuning, GUI verification, and hardware integration.

9.2. ITERATION 2: MECHANICAL FRAME, PCB, AND MOTOR CONTROL

The second iteration focused on the physical realization of the robot. The mechanical frame was designed and 3D printed. The design uses a face-centered mechanism in which six stepper motors rotate the six cube faces. A custom PCB was designed to host or connect the stepper motor driver modules, organize the GPIO interface, and centralize power distribution. Motor-control scripts were implemented in Python so that each solver move could be translated into a step and direction pulse sequence.

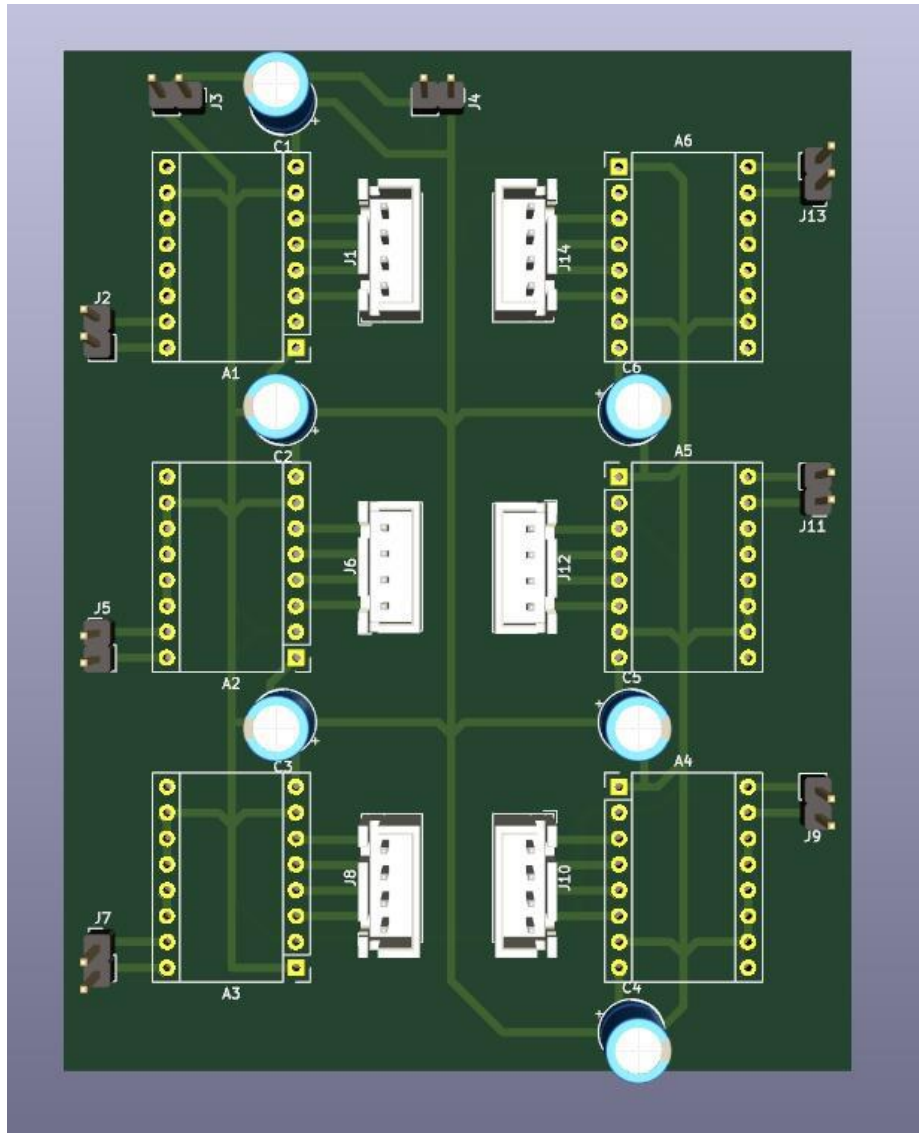


Figure 7. Custom PCB layout and transition from breadboard wiring to centralized motor control

9.2.1. TESTING AND RESULTS

Before full PCB integration, motor rotation was tested on a breadboard test bench. The interim measurements showed a mean angular error of 1.2 degrees for a 90-degree clockwise turn, 1.5 degrees for a 90-degree counter-clockwise turn, and 2.1 degrees for a 180-degree turn. The 3D-printed parts were also inspected for fit, and the main structural components were found to be suitable for assembly.

9.2.2. EVALUATION

The tests confirmed that the motor-control principle was correct. The remaining weakness was synchronization: the system still needed a final GUI layer, a reliable state transition between scan, solve, and execute, and a method to demonstrate that the robot could solve a cube after it had been automatically mixed by the robot itself. This led to the final iteration.

9.3. ITERATION 3: FINAL GUI, RANDOM MIX, AND END-TO-END OPERATION

The third iteration completed the final user workflow. The GUI was integrated with the color-detection, solver, motor-control, and state-memory modules. The user can display the detected cube state, manually correct colors if necessary, solve the cube, execute motor moves, stop the operation, or use the new Random Mix command. Random Mix was added as a final validation feature: the robot first scrambles the cube using a random sequence, then it forgets the stored state and previous solution. When Solve is pressed, the system must solve the current physical cube again.

9.3.1. TESTING AND RESULTS

The final workflow was tested by operating the GUI, executing random motor sequences, clearing the saved state, and running the solve pipeline again. The main observation was that the GUI and state-reset behavior prevented the program from relying on the previous random sequence. This means the solve action remained dependent on the current cube state rather than on a stored inverse scramble. The final poster results reported that the system successfully solves a 3 x 3 x 3 cube using six NEMA 17 stepper motors and a Raspberry Pi 4B, with HSV-based color detection and GUI-supported manual verification.

9.3.2. EVALUATION

The final iteration significantly improved the demonstration value of the project. The Random Mix feature proves that the robot is not only playing back a prepared solution; it can create a new scramble, reset its memory, and solve again from the updated physical configuration. The remaining limitation is that the color detection still depends on lighting stability and user verification. Closed-loop motor feedback and adaptive lighting calibration are recommended for future versions.

10. FINAL DESIGN

The final design is a Raspberry Pi 4B-based autonomous Rubik's Cube solving robot. The data path starts from the camera image and ends with physical face rotations. The camera captures cube-face images; OpenCV extracts sticker colors; the GUI displays and validates the state; the Kociemba solver computes the move list; the move parser converts moves into motor commands; the custom PCB and A4988 drivers distribute the control signals; and six NEMA 17 motors rotate the corresponding cube faces.

The final software modules are: image acquisition, HSV segmentation, ROI mapping, GUI display, manual color correction, cube-state validation, Kociemba solving, Random Mix generation, state-memory reset, move parsing, GPIO pulse generation, and stop/error handling. The final hardware modules are: Raspberry Pi 4B, camera, custom PCB, six A4988 drivers, six NEMA 17 motors, power distribution, 3D-printed frame, cube adapters, connectors, and wiring.

The Random Mix command is a central final-design feature. The command generates a random legal move sequence, executes it with the motors, clears the old cube state and solution, and returns the GUI to a ready state. The next Solve command performs a new solve cycle. This design choice makes the demonstration more convincing because the system cannot solve by remembering the random scramble sequence.

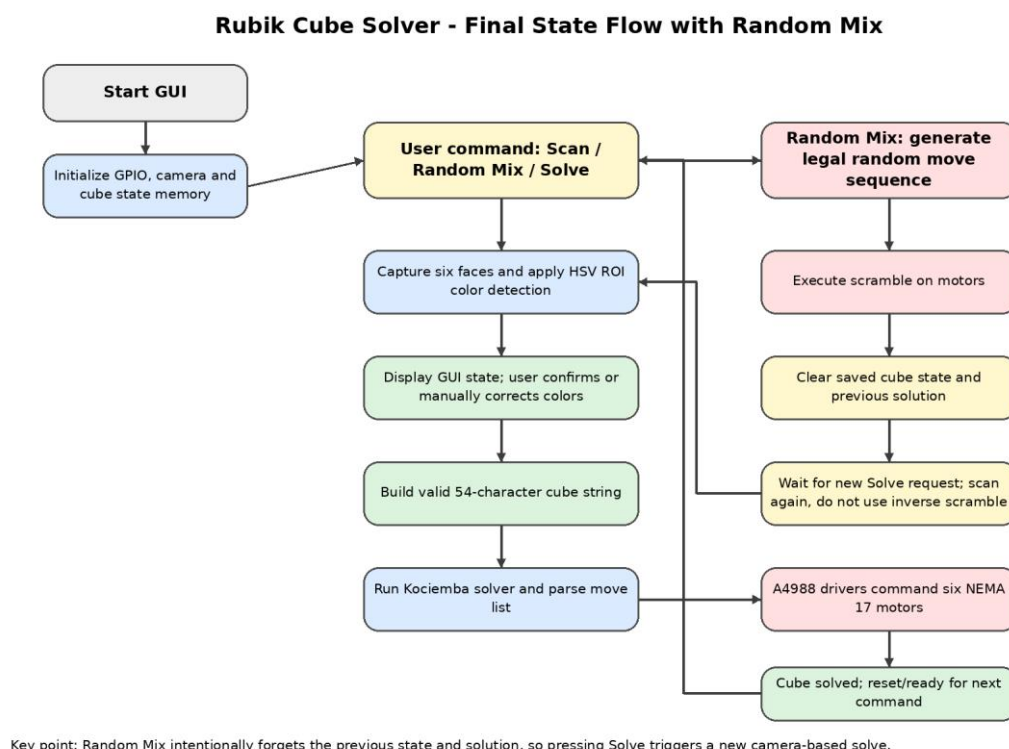


Figure 8. Final state flowchart with Random Mix and state reset behavior

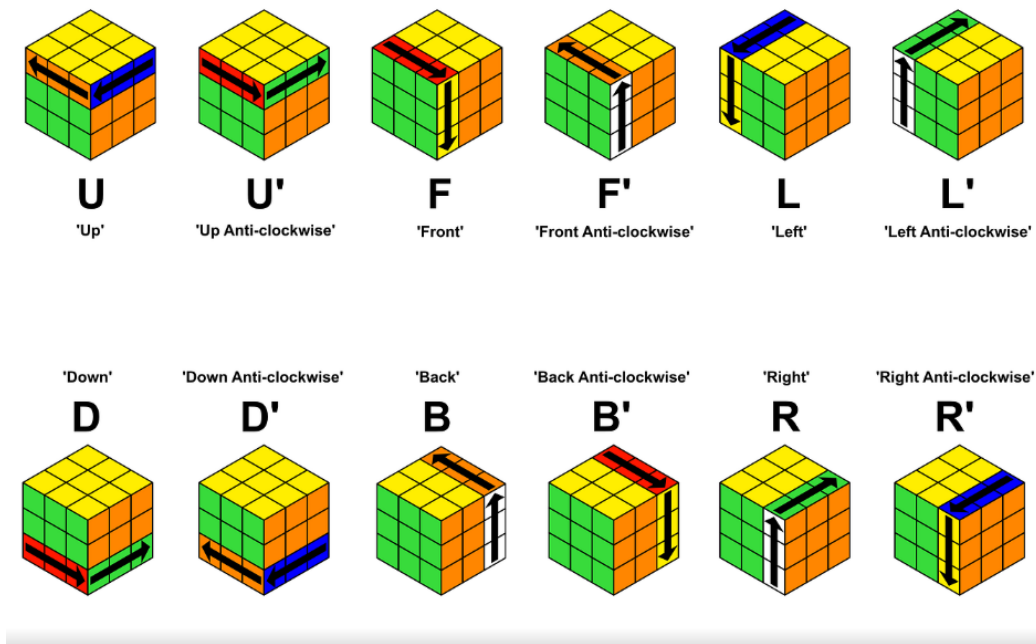


Figure 9. Standard Rubik's Cube move notation used by the solver and motor parser

The final design meets the core project objectives. It integrates computer vision, motion planning, embedded control, motor driver electronics, and a mechanical frame. Its strongest features are modularity, low-cost implementation, and the GUI-supported correction workflow. Its main weaknesses are sensitivity to lighting conditions, open-loop stepper operation, and the need for careful mechanical alignment. These weaknesses are acceptable for a graduation-project prototype and provide clear directions for future improvement.

10.1. MEETING THE CONSTRAINTS AND ENGINEERING STANDARDS

The final design meets the functional constraint of scan-solve-execute operation by using the Raspberry Pi as a single coordination unit. It meets the mechanical constraint through a six-motor frame that allows each cube face to be rotated independently. It meets the cost constraint by relying on affordable and accessible components rather than industrial robot hardware. The PCB improves reliability and follows the PCB-design intent of IPC-2221 by replacing unstable breadboard wiring with organized routing and connectors. Low-voltage safety principles were addressed by separating logic and motor power, limiting the prototype to low-voltage operation, securing the cube in a frame, and avoiding user contact with moving parts during operation.

The software was organized into modules in line with software life-cycle good practice. Color acquisition, GUI verification, solver logic, random mixing, state reset, and motor-control functions can be tested separately. The final Random Mix behavior also satisfies a validation constraint: solving must be based on the actual cube state after scrambling rather than on a stored solution.

10.2. COST ANALYSIS

The cost analysis below gives estimated prototype costs. Actual prices may vary depending on supplier, exchange rate, and previously available lab materials.

Table 3. Estimated cost analysis of the final prototype

Component	Quantity	Estimated unit cost (USD)	Estimated subtotal (USD)
Raspberry Pi 4B	1	85	85
Camera / webcam module	1	15	15
NEMA 17 stepper motor	6	12	72
A4988 stepper motor driver	6	3	18
Custom PCB manufacturing	1	25	25
12 V motor power supply	1	20	20
3D-printed PLA frame and adapters	1 set	30	30
Connectors, wiring, screws, couplers	1 set	30	30
Rubik's Cube	1	8	8
Total estimated cost			303

The estimated total cost is approximately 303 USD. Considering that the prototype includes computer vision, a graphical interface, six independent motors, motor drivers, a custom PCB, and a mechanical frame, the cost is reasonable for an educational mechatronics platform.

11. TEAMWORK

The project was completed by Özge Erdem and Görkem Yüksel. Both members contributed to the overall system design, debugging, testing, and final demonstration. The individual contributions are summarized below.

- Özge Erdem (2210357057): Worked on the computer-vision pipeline, HSV color calibration, GUI workflow, manual correction logic, Random Mix state-reset behavior, system testing, report preparation, and final documentation.
- Görkem Yüksel (2210357094): Worked on mechanical design, 3D-printed frame preparation, PCB design and motor-driver organization, stepper motor control tests, system integration, poster preparation, and final demonstration support.
- Joint work: Kociemba solver integration, cube-state validation, move parser debugging, end-to-end testing, final result evaluation, and preparation of presentation materials.

12. COMMENTS AND CONCLUSIONS

This report presented the final status of the Rubik's Cube Solver graduation project. The project started as a software-oriented solver concept and was completed as a functional mechatronic prototype. The final design uses a Raspberry Pi 4B, camera-based HSV color detection, Kociemba solving, GUI-based manual verification, a custom PCB, six A4988 motor drivers, six NEMA 17 stepper motors, and a 3D-printed mechanical frame. The design achieved the main objective of building an autonomous system that can scan, solve, and physically manipulate a standard 3 x 3 x 3 Rubik's Cube.

The most important final improvement was the Random Mix feature. With this feature, the robot can scramble the cube by itself, clear the stored cube state and previous solution, and then solve the cube again when the Solve command is pressed. This addition strengthens the demonstration because it verifies that the system relies on the current physical cube state rather than a memorized inverse scramble.

The final design is successful as a low-cost educational prototype. It demonstrates the integration of computer vision, embedded programming, motor control, PCB design, and mechanical prototyping. The remaining limitations are mostly related to lighting sensitivity, open-loop motor control, and mechanical tolerance. Future work should focus on adaptive lighting compensation, automatic camera calibration, closed-loop face-position feedback using encoders or limit sensors, improved acceleration/deceleration profiles, and a more rigid enclosure for repeatable alignment.

REFERENCES

- [1] H. Kociemba, "Two-phase algorithm for solving Rubik's Cube," Cube Explorer Documentation, 1997. Available: <https://kociemba.org/cube.htm>
- [2] H. Kociemba, "hkociemba/RubiksCube-TwophaseSolver," GitHub, Jan. 08, 2021. Available: <https://github.com/hkociemba/RubiksCube-TwophaseSolver>
- [3] D. Singmaster, Notes on Rubik's Cube. Enslow Publishers, 1981.
- [4] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [5] Raspberry Pi Foundation, "Raspberry Pi Documentation - Hardware and GPIO," 2024. Available: <https://www.raspberrypi.com/documentation/>
- [6] Allegro MicroSystems, "A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection," Datasheet.
- [7] C. T. Huang et al., "Design and control of a Rubik's Cube solving robot," IEEE Transactions on Automation Science and Engineering, vol. 14, no. 3, pp. 1385-1395, Jul. 2017.
- [8] IPC, IPC-2221: Generic Standard on Printed Board Design.
- [9] IEC, IEC 62368-1: Audio/video, information and communication technology equipment - Safety requirements.
- [10] ISO/IEC/IEEE, ISO/IEC/IEEE 12207: Systems and software engineering - Software life cycle processes.
- [11] Autodesk, "Tinkercad - 3D Design and Electronics Modeling," 2024. Available: <https://www.tinkercad.com/>
- [12] Bambu Lab, "MakerWorld - 3D Model Community and Printing Resources," 2024. Available: <https://makerworld.com/>